

Profiling & Alleviating Performance Bottlenecks in Zeek

Jyotsna Arunkumar

Arnav Batra

Matthew Sickler

The University of Texas at Austin
Austin, USA

ABSTRACT

(The two-page proposal should describe the problem, its relationship to the course, the proposed work, the responsibilities of each team member, the resources required, and how the outcome will be evaluated. Your proposal should convince us that the project is relevant, technically meaningful, feasible within one semester, and supported by a concrete evaluation plan.

Engineering-track proposals should identify the intended users and their current workflow, survey existing tools, explain the missing capability, define the proposed interface and implementation, and describe how you will demonstrate correctness and usefulness. The proposal must also explain how the project can be developed and evaluated without specialized hardware or a multi-GPU cluster.)

1 RELATED WORK

(include summary of related work and existing gaps in research here)

2 PROBLEM & MOTIVATION

First developed under the name "Bro" in the 1990s, Zeek is an open-source network security monitoring platform that passively analyzes real-time network traffic, performing actions including protocol parsing, connection tracking, event generation, scripting, and logging. According to the Zeek Project website, it has over 10,000 deployments worldwide [1].

However, Zeek's packet-processing cost can vary substantially with traffic rate, packet size, flow churn, protocol mix, enabled analyzers, and scripts. Within a single Zeek worker, these costs can create processing bottlenecks, which may lead to incomplete visibility into network traffic as packets are dropped from processing queues. In deployments with multiple workers, which are typically assigned packets via flow hashing, this variable cost can also lead to load imbalances across workers. As a result, the actual processing capacity of a worker cluster may be lower than the theoretical maximum, as seen in metrics including throughput and packet drops.

("identify the intended users and their current workflow, survey existing tools, explain the missing capability")

3 PROPOSED WORK

("define the proposed interface and implementation")

4 EVALUATION

We will evaluate the project using controlled workloads that vary four traffic characteristics: packet rate, packet size, flow churn, and protocol mix. Profiling these variables may help explain why some flows are more expensive to process than others.

Our evaluation will focus on whether these workload characteristics contribute to bottlenecks and load imbalances, and how our proposed mitigation improves performance.

Our main metrics will be:

- packet-processing throughput,
- packet-drop rate,
- CPU utilization,
- worker load imbalance,
- memory consumption
- any overhead introduced by the proposed mitigation

4.1 75% Goal

We will characterize how Zeek performance and worker imbalance change by running workloads with different packet rates, packet sizes, flow churn, and protocol mixes. Using these experiments, we expect to narrow down to one reproducible performance bottleneck.

We will then implement a simple mitigation that addresses the identified bottleneck. For example, if worker imbalance is the targeted issue, we will implement cost-aware flow-assignment to send new flows to a less-loaded Zeek worker.

We will compare the proposed mitigation against Zeek's existing behavior under the same workload. We will consider this goal successful if the mitigation demonstrates a measurable performance improvement, like reduced worker imbalance, fewer packet drops, lower CPU utilization, or higher throughput.

4.2 100% Goal

We will design and implement an improved mitigation based on the bottleneck identified during profiling and on the results of the 75% implementation. For worker imbalance, we will iterate on our cost-aware flow-assignment strategy to

consider the estimated processing cost of a new flow and the current load of each Zeek worker while preserving flow affinity. For other bottlenecks, we will design an improved mitigation.

We will evaluate the improved mitigation across different types of network traffic, comparing it against Zeek’s existing performance. We will measure metrics such as throughput, packet drops, CPU utilization, worker-load imbalance, memory consumption, and overhead. Using these metrics, we will characterize the observed benefits and possible limitations of the mitigation.

4.3 125% Goal

We will extend the 100% solution with a more dynamic implementation that responds to changes in workload behavior. One possible extension is to predict a new flow’s processing cost using information from its first few packets (e.g. packet size, header metadata, transport protocol). We will evaluate whether this prediction improves worker assignment compared with the 100% solution.

For other bottlenecks, the 125% goal will extend the existing mitigation, possibly by identifying unusually expensive flows or dynamically adding/removing workers.

5 REQUIRED RESOURCES

The proposed work does not require specialized hardware or a multi-GPU cluster.

Our main resources will include:

- **CloudLab** for compute resources
- **tcpreplay** for existing packet captures
- **iperf3** for generating traffic with variable packet sizes and flow patterns
- **Linux perf** for profiling CPU usage
- **zeekctl top** for measuring per-worker CPU and memory utilization
- **Zeek’s Prometheus tool** for logging per-worker CPU metrics during automated experiments

6 TEAM MEMBER RESPONSIBILITIES

(Replace this text with the body of your paper.)

REFERENCES

- [1] Zeek Project. [n. d.]. *Zeek*. <https://zeek.org/>